

LightCCI Compare 矩阵目录重构与 Joint NMF 邻接矩阵版 Codex Plan

0. 目标

在现有 MIGNet / causality 代码中，为 light_cci 网络新增一组 compare 对比实验 PIJ 方法。实验矩阵的横轴是特征构造方式，纵轴是从特征到 PIJ 的方法。矩阵中的每一个格子都必须是一个独立的 PIJ method 文件，便于单独注册、单独运行、单独调试、单独记录输出。

本轮特别明确三件事：

1. Joint NMF 特征 N 直接使用 LightCCI / COMMOT 输出得到的邻接矩阵，尊重原来源代码中的 temporal joint NMF 流程。
2. Laplacian-HKS 特征 L 和 Joint NMF 特征 N 的图输入都来自 COMMOT / CCI 的输出，例如 *_CCI_total.npz。
3. 需要重构 mignet_ce/pij/ 目录：旧 PIJ 方法收进 legacy/，本轮 compare 矩阵方法放进 compare/。

1. 需要先提醒 Codex 的数据事实

本机来源文件中有 sample data，可以先用于查看 COMMOT / CCI 输出的文件结构和矩阵形态。Codex 在写代码前应先检查 sample data 中的 .npz、index 文件和目录命名，例如：

```
E1S1_domain_factory_sample/cci/louvain_k150/louvain150_heart_11.5_CCI_total.npz
E1S1_domain_factory_sample/cci/louvain_k150/louvain150_heart_11.5_index.tsv
E1S1_domain_factory_sample/cci/louvain_k150/louvain150_heart_11.5_COMMOT_by_LR/*.npz
E1S1_domain_factory_sample/cci/seurat_k40/seurat_heart_11.5_CCI_total.npz
E1S1_domain_factory_sample/cci/seurat_k40/seurat_heart_11.5_index.tsv
E1S1_domain_factory_sample/cci/seurat_k40/seurat_heart_11.5_COMMOT_by_LR/*.npz
```

本机 sample data 只用于理解文件格式、shape、nnz、index 对齐方式和 fallback 逻辑；正式实验代码仍然要放到服务器项目中运行。服务器真实数据路径沿用现有代码已经使用过的路径体系，例如：

```
/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory
/home/jovyan/work/2026 Causality
```

Codex 不要把本机 sample 路径硬编码到正式代码中。正式代码应通过 --data-root、config 或现有 path resolver 获取服务器数据路径。

2. 目录结构重构要求

当前 mignet_ce/pij/ 下面已经有很多历史 PIJ 文件，例如 expr_ot.py、joint_nmf.py、laplacian.py、sr_ot.py、slat.py 等。这些旧方法和本轮 compare_* 矩阵不是同一个实验批次，不

能继续全部混在同一层目录里。

请把目录改成下面的结构：

```
mignet_ce/pij/  
  __init__.py  
  base.py  
  registry.py  
  legacy/  
    __init__.py  
    development_ot.py  
    energy_entropy_ot.py  
    energy_ot.py  
    expr_ot.py  
    expr_pseudotime_sr_energy_ot.py  
    expr_pseudotime_sr_energy_spatial_ot.py  
    expr_pseudotime_sr_ot.py  
    expr_pseudotime_sr_spatial_ot.py  
    joint_nmf.py  
    laplacian.py  
    pseudotime_expression_ot.py  
    pseudotime_ot.py  
    pseudotime_spatial_ot.py  
    pure_expression_ot.py  
    slat.py  
    spatial_ot.py  
    sr_expression_ot.py  
    sr_ot.py  
    sr_spatial_ot.py  
    three_dot.py  
    velocity_ot.py  
    _developmental_ot_components.py  
    _ot_common.py  
  compare/  
    __init__.py  
    common.py  
    features.py  
    distances.py  
    sparse_ot.py  
    compare_E_cos.py  
    compare_E_kl.py  
    compare_E_sot.py  
    ...
```

注意：

1. `base.py`、`registry.py` 和顶层 `__init__.py` 建议先保留在 `mignet_ce/pij/` 根目录, 避免大面积破坏现有导入。
2. 旧 PIJ 方法实现文件移动到 `mignet_ce/pij/legacy/` 后, 要同步修改 `import` 路径。
3. 如果 `_ot_common.py` 或 `_developmental_ot_components.py` 只服务旧方法, 就一起放进 `legacy/`; 如果 `compare` 也需要其中逻辑, `Codex` 要么在 `compare` 内显式复用, 要么把真正通用的函数上移到更中性的工具层, 但不要为了本轮任务做过度重构。
4. `registry.py` 需要同时注册 `legacy` 方法和 `compare` 方法。已有 `method name` 不能被破坏。
5. 所有 `compare` 方法都放在 `mignet_ce/pij/compare/` 下面。

3. `compare` 子目录里的命名规则

因为已经有 `compare/` 子目录, 所以公共工具文件不要再加 `compare_` 前缀。公共工具文件就叫:

```
common.py
features.py
distances.py
sparse_ot.py
```

这些文件只放跨矩阵格子确实需要复用的逻辑, 不注册为 PIJ method。

矩阵格子文件仍然使用 `compare_` 前缀, 因为每一个格子就是一个独立 PIJ method 文件。例如:

```
compare_E_cos.py
compare_N_cos.py
compare_L_sot.py
compare_L_Sr_kl.py
```

不要再新增 `compare_adjacency_nmf.py` 这类 NMF 公共文件。邻接矩阵版 Joint NMF 是 N 特征的 `specific` 逻辑, 不需要单独 `generalize` 成公共模块。实现时可以在涉及 N 的格子文件中写清楚, 或者在 `features.py` 里作为 N 特征分支实现, 但不要新建一个专门的 NMF 公共工具文件。

4. 实验矩阵

基础特征:

- E: expression feature。
- N: adjacency-based Joint NMF node-factor feature。
- L: Laplacian-HKS feature。
- Sr: SR / developmental feature。

第一版特征列:

```
E
N
L
Sr
```

E_N
E_L
E_Sr
N_L
N_Sr
L_Sr

PIJ 生成方法:

cos: cosine kernel 到 PIJ
kl: feature-level KL kernel 到 PIJ
sot: sparse semi-relaxed OT 到 PIJ

共 $10 \times 3 = 30$ 个核心格子。

5. 必须新增的 compare PIJ 文件

在 mignet_ce/pij/compare/ 下新增以下 30 个文件:

compare_E_cos.py
compare_E_kl.py
compare_E_sot.py
compare_N_cos.py
compare_N_kl.py
compare_N_sot.py
compare_L_cos.py
compare_L_kl.py
compare_L_sot.py
compare_Sr_cos.py
compare_Sr_kl.py
compare_Sr_sot.py
compare_E_N_cos.py
compare_E_N_kl.py
compare_E_N_sot.py
compare_E_L_cos.py
compare_E_L_kl.py
compare_E_L_sot.py
compare_E_Sr_cos.py
compare_E_Sr_kl.py
compare_E_Sr_sot.py
compare_N_L_cos.py
compare_N_L_kl.py
compare_N_L_sot.py
compare_N_Sr_cos.py

```
compare_N_Sr_kl.py
compare_N_Sr_sot.py
compare_L_Sr_cos.py
compare_L_Sr_kl.py
compare_L_Sr_sot.py
```

每个文件都定义一个独立 class 或 function, 并被 `mignet_ce/pij/registry.py` 注册成独立 `pij_method`。method name 与文件名保持一致, 例如:

```
compare_E_cos
compare_N_sot
compare_L_Sr_kl
```

6. COMMOT / CCI 输出读取要求

N 和 L 都需要读取 LightCCI / COMMOT 输出得到的邻接矩阵。第一优先级读取聚合矩阵:

```
*_CCI_total.npz
*_index.tsv
```

其中 `*_CCI_total.npz` 是邻接矩阵, `*_index.tsv` 用于确认矩阵行列和节点顺序。Codex 必须先在 sample data 中确认 .npz 的实际 sparse 格式、shape、nnz 和 index 文件列名。

如果 `*_CCI_total.npz` 不存在, 才回退到 LR-pair 级别文件:

```
*_COMMOT_by_LR/*.npz
```

回退时需要把多个 LR-pair 通讯矩阵求和, 得到:

$$A_{\{\ell, t\}^{\{c'\}}} = \sum_{q \in Q_{\{\ell, t\}}} M_{\{\ell, t\}^{\{q\}}}$$

这里, $A_{\{\ell, t\}^{\{c'\}}}$ 是进入 LightCCI compare 特征的邻接矩阵; $M_{\{\ell, t\}^{\{q\}}}$ 是第 q 个 LR pair 的 COMMOT 通讯矩阵; $Q_{\{\ell, t\}}$ 是该层级、该时间点可用的 LR pair 集合。

7. Joint NMF 邻接矩阵版数学定义

对一个样本组 $g=(c', t, \ell)$, 读取 LightCCI / COMMOT 聚合邻接矩阵:

$$A_{\{\ell, t\}^{\{c'\}}} \in \mathbb{R}_{+}^{n_t \times n_t}$$

这里, $A_{\{\ell, t\}^{\{c'\}}}$ 是该器官、时间点、层级的非负邻接矩阵; n_t 是该时间点该层级节点数。cell / spot 层的边主要来自 CCI, domain 层的边来自 DDI 或聚合 CCI/DDI, gene 层的边来自 gene x gene GRN。

Joint NMF 特征 N 直接使用邻接矩阵作为输入:

$$X_t^N = A_{\{\ell, t\}^{\{c'\}}}$$

调用原来源代码中的 temporal joint NMF 数学过程:

$$\min_{\{W_t^N\}, H^N} \sum_t \|X_t^N - W_t^N H^N\|_F^2$$

输出节点因子特征：

$$x_i^N = W_t^N(i, :)$$

这里， x_i^N 是节点 i 的 adjacency / connectivity latent factor，表示该节点在若干个低秩邻接模式上的负载。它与 Laplacian-HKS 的区别是：Joint NMF 是低秩邻接模式表征，Laplacian-HKS 是谱扩散拓扑表征。

第一版不要把 N 改成 LR-pair profile，不拼接 $A_t.T$ ，不做 padding，不做 truncation。若不同时间点 $X_t^N.shape[1]$ 不一致，给出清晰 diagnostic 或 structured skip result。

8. features.py 的职责边界

mignet_ce/pij/compare/features.py 只负责统一特征构建入口和组合拼接。建议提供类似接口：

```
def build_compare_features(context, cfg, feature_keys, side):
    ...
```

其中：

- E 分支：读取并聚合表达特征。
- N 分支：读取 COMMOT / CCI 邻接矩阵，并按 $X_t^N = A_t$ 调用现有 temporal joint NMF 逻辑。不要另建 compare_adjacency_nmf.py。
- L 分支：读取同一类 COMMOT / CCI 邻接矩阵，做 Laplacian-HKS。
- Sr 分支：读取 development feature table 中的 SR 或发育状态量。

组合特征时，先对每个基础特征分别标准化，再拼接：

$$f_i^{\mathcal{A}} = \operatorname{concat}_{a \in \mathcal{A}} \left(\sqrt{w_a} \widehat{x}_i^a \right)$$

不要先拼接再统一标准化，避免高维特征支配距离。

9. common.py、distances.py、sparse_ot.py 的职责

common.py：

- 定义 feature key、method key、metadata schema。
- 提供 blockwise pair iteration。
- 提供单位对齐、行归一化、sparse matrix 导出辅助。
- 不包含 NMF 专属逻辑。

distances.py：

- blockwise cosine distance。
- feature-level KL cost。
- kernel 到 row-normalized PIJ。
- 避免一次性构造超大 dense cost matrix。

sparse_ot.py:

- 根据当前特征组合的 cosine distance 构造候选集合。
- 只在候选边上保存 cost。
- 实现 sparse semi-relaxed Sinkhorn。
- 导出 raw transport 和 row-normalized PIJ。

10. 三类 PIJ 方法实现要求

10.1 cos

计算当前特征组合的 cosine distance:

$$d_{\mathcal{A}}(i, j) = 1 - \frac{\langle f_i^{\mathcal{A}}, f_j^{\prime \mathcal{A}} \rangle}{\|f_i^{\mathcal{A}}\|_2 \|f_j^{\prime \mathcal{A}}\|_2 + \delta}$$

转成 kernel:

$$K_{ij} = \exp(-d_{\mathcal{A}}(i, j) / \tau)$$

按 source 行归一化得到 PIJ。

10.2 kl

将特征转成概率分布:

$$\pi_i^{\mathcal{A}} = \operatorname{softmax}(f_i^{\mathcal{A}} / \beta)$$

计算 feature-level KL:

$$D_{ij} = \operatorname{KL}(\pi_i^{\mathcal{A}} \| \pi_j^{\prime \mathcal{A}})$$

转成 kernel 后按 source 行归一化。

10.3 sot

矩阵版 sparse semi-relaxed OT 的预成本只使用当前特征组合的 cosine distance:

$$B_{ij}^{\mathcal{A}} = d_{\mathcal{A}}(i, j)$$

候选集合:

$\Omega = \text{TopK}_{\text{source}}(B) \cup \text{TopK}_{\text{target}}(B)$

稀疏 cost:

$C_{ij}^{\mathcal{A}} = \text{normalize}(B_{ij}^{\mathcal{A}}), \quad (i,j) \in \Omega$

优化:

$$\min_P \sum_{(i,j) \in \Omega} P_{ij} C_{ij}^{\mathcal{A}} + \epsilon \sum_{(i,j) \in \Omega} P_{ij} (\log P_{ij} - 1) + \gamma \text{KL}(P \| \mathbf{1} | \mathbf{a})$$

$\text{s.t.} \quad P^{\text{top}} \mathbf{1} = \mathbf{b}$

导出 raw sparse transport、row-normalized PIJ、source mass diagnostics 和 OT convergence diagnostics。

11. registry 修改要求

修改 mignet_ce/pij/registry.py:

1. 从 mignet_ce.pij.legacy.* 导入旧 PIJ 方法, 并保持原 method name 不变。
2. 从 mignet_ce.pij.compare.* 导入 30 个 compare 方法。
3. 新 compare method name 使用文件名去掉 .py 后的名字, 例如 compare_N_sot。
4. 不要覆盖已有 joint_nmf、laplacian、expr_ot 等旧方法。
5. 加一个 registry smoke test, 确认 legacy 和 compare 方法都能被发现。

12. 输出文件要求

每个矩阵格子、每个 organ、每个 level pair、每个 time pair 输出到独立目录, 目录名中包含 method name。

基础输出:

```
metadata.json
feature_source.json
units_source.csv
units_target.csv
features_source.npy
features_target.npy
cost_or_kernel_diagnostics.json
pij_sparse.npz
pij_row_normalized_sparse.npz
```

sot 额外输出:

candidate_edges.parquet
 cost_sparse.npz
 pij_transport_sparse.npz
 source_mass_diagnostics.csv
 ot_convergence.json

N 相关格子额外输出:

joint_nmf_shapes.json
 joint_nmf_H.npy
 joint_nmf_W_source.npy
 joint_nmf_W_target.npy
 joint_nmf_diagnostics.json

13. Profile 与进度条

新增 profile 模式或脚本:

scripts/profile_lightcci_compare_matrix.py

profile 内容:

- 本机 sample data 中 *_CCI_total.npz 和 *_index.tsv 的格式检查结果。
- 服务器目标数据中每个时间点的 adjacency shape、nnz、文件大小。
- N 特征是否满足 temporal joint NMF 的列数一致要求。
- L 特征 eigensolver 规模。
- E 与 Sr 特征维度。
- 每个 time pair 的 $|S| \times |T|$ 。
- sot 候选边数量估计。
- dense PIJ 内存估计。
- sparse OT 内存估计。

进度条层级:

1. organ / layer pair / time pair / compare method。
2. feature 构建进度。
3. COMMOT / CCI .npz 读取和 adjacency 构建进度。
4. blockwise distance 进度。
5. sparse OT Sinkhorn iteration 进度。

14. 验证标准

最小验证必须包括:

1. sample data 中的 *_CCI_total.npz 能被读取, 并且 shape、nnz、index 对齐检查能输出。
2. legacy 方法移动到 mignet_ce/pij/legacy/ 后, 原 method name 仍然可注册、可导入。

3. compare 方法在 mignet_ce/pij/compare/ 下可注册、可导入。
4. compare_E_cos 能运行。
5. compare_N_cos 在列数一致时能运行。
6. compare_N_cos 在列数不一致时给出清晰 diagnostic 或 skip result。
7. compare_L_sot 的预成本只来自 L 的 cosine distance。
8. compare_L_Sr_sot 的预成本只来自 L+Sr 拼接特征的 cosine distance。
9. 30 个 compare method 都能被 registry 发现。
10. 每个格子的 metadata 能准确记录 feature 组合、PIJ 方法、输入 adjacency 来源和 sample/server 路径来源。

15. 不要做的事情

1. 不要把 N 改成 LR-pair profile 主方案。
2. 不要新增 compare_adjacency_nmf.py 作为 NMF 公共工具文件。
3. 不要在第一版自动拼接 A_t.T。
4. 不要在第一版自动 padding 或 truncation adjacency。
5. 不要让一个文件通过参数切换所有 feature。
6. 不要把综合版 Laplacian + SR + spatial + auxiliary graph 混进核心矩阵版 OT。
7. 不要把本机 sample data 路径硬编码到服务器正式运行代码中。
8. 不要破坏已有 legacy PIJ method name。

16. 推荐实现顺序

1. 先检查 sample data 里的 *_CCI_total.npz、*_index.tsv 和 *_COMMOT_by_LR/*.npz, 记录 shape、nnz 和 index 对齐方式。
2. 重构 mignet_ce/pij/: 新建 legacy/ 和 compare/。
3. 把旧 PIJ 方法文件移动到 legacy/, 修复 imports, 确保原 method name 不变。
4. 在 compare/ 下新增 common.py、features.py、distances.py、sparse_ot.py。
5. 在 features.py 内实现 E、N、L、Sr 的统一构建入口, 其中 N 使用 $X_t^N = A_t$ 。
6. 先实现并注册 4 个 smoke test 文件:
 - compare_E_cos.py
 - compare_N_cos.py
 - compare_L_sot.py
 - compare_L_Sr_sot.py
7. 跑 registry smoke test, 确认 legacy 和 compare 方法都能导入。
8. 跑 profile, 确认 sample data 和服务端目标路径的数据规模估计。
9. 批量补齐剩余 26 个 compare 格子文件。
10. 注册全部 30 个 compare method。
11. 输出矩阵运行命令模板。
12. 最后补充 README 或实验日志说明。